

Constraint Programming on Integers

A practical tour, with NuCS

Yan Georget — 60 minutes

What you'll get out of this hour

- **Part 1** — Why we need CP: NP-hard problems, the search-space explosion
- **Part 2** — The theory: CSP, propagation, consistency, search
- **Part 3** — Real problems: Queens, Sudoku, Golomb, TSP — in NuCS
- **Part 4** — Inside a solver: propagators, the propagation loop, backtracking

No prior CP knowledge assumed. We stay on **integer variables** throughout.

Part 1 — Why constraint programming?

A familiar pain: NP-completeness

A problem is **NP** if a candidate solution can be checked in polynomial time.

A problem is **NP-hard** if every NP problem reduces to it.

NP-complete = $\text{NP} \cap \text{NP-hard}$.

Classic NP-complete decision problems you've likely met:

- **SAT**: is this Boolean formula satisfiable?
- **3-coloring**: can this graph be 3-colored?
- **Hamiltonian cycle**: does this graph have a tour visiting every node once?
- **Subset sum, bin packing, scheduling with precedences, ...**

No known polynomial-time algorithm. But we still need answers.

The naive approach blows up fast

8-queens: place 8 queens on a 8×8 board, no two attacking.

- Naive enumeration: $\binom{64}{8} \approx 4.4 \cdot 10^9$ placements.
- One-queen-per-column: $8^8 \approx 1.7 \cdot 10^7$.
- Permutations (one per column, one per row): $8! = 40\,320$.
- A good CP model: solves it in **microseconds**, exploring ~100 nodes.

The art is **encoding** the problem so the solver can prune aggressively.

Where CP fits among solvers

Paradigm	Variables	Strength
SAT	Boolean	Huge industrial formulas, conflict learning
MIP	Continuous + integer	Linear constraints, strong LP relaxation
SMT	Boolean + theories	Verification, symbolic execution
CP	Integer (mostly), finite	Expressive global constraints , combinatorial puzzles, scheduling

CP shines when constraints are **structured and non-linear**: `all-different`, `circuit`, `cumulative`, `element`, ...

A CP problem in one breath

A **Constraint Satisfaction Problem (CSP)** is a triple $\langle X, D, C \rangle$:

- $X = \{x_1, \dots, x_n\}$ — **variables**
- $D = \{D_1, \dots, D_n\}$ — **domain** for each variable (here: finite sets of integers)
- $C = \{c_1, \dots, c_m\}$ — **constraints**, each restricting a subset of variables

A **solution** assigns each x_i a value $v_i \in D_i$ such that every $c \in C$ holds.

If we also want **min** or **max** of an objective variable, it becomes a **COP** (Constraint Optimization Problem).

Teaser: N-Queens in NuCS

```
from nucs.problems.problem import Problem
from nucs.propagators.propagators import ALG_ALLDIFFERENT
from nucs.solvers.backtrack_solver import BacktrackSolver

class QueensProblem(Problem):
    def __init__(self, n: int):
        super().__init__([(0, n - 1)] * n) # n variables, domain 0..n-1
        self.add_propagator(ALG_ALLDIFFERENT, range(n)) # rows
        self.add_propagator(ALG_ALLDIFFERENT, range(n), range(n)) # diag ↘
        self.add_propagator(ALG_ALLDIFFERENT, range(n), range(0, -n, -1)) # diag ↙

solver = BacktrackSolver(QueensProblem(8))
print(next(solver.solve()))
```

Three lines of model. The rest is the solver's job.

Part 2 — Theory: how a CP solver thinks

Two ingredients: propagation and search

Solving a CSP is the interleaving of:

1. **Propagation** — for each constraint, remove from each variable's domain values that cannot participate in any solution of that constraint. Repeat until nothing changes.
2. **Search** — when propagation can't decide, **branch**: pick a variable, split its domain, recurse. On failure, **backtrack**.

Propagation alone rarely solves anything.

Search alone explodes.

Together they're powerful.

Domains, in NuCS specifically

NuCS uses **bound consistency**: each domain is represented as an **interval** $[\min, \max]$, not a full set.

```
super().__init__([(0, n - 1)] * n)    # n domains, each = interval [0, n-1]
```

- Cheap: 2 integers per variable.
- A variable is **ground** when $\min = \max$.
- We lose the ability to remove "holes" inside an interval — but we gain speed.
- Trade-off well known: bound consistency vs. arc consistency.

Consistency levels (informally)

For a constraint c on variables $x_1, \dots, x_k$:

- **Node consistency** — values in D_i satisfy any unary constraint on x_i .
- **Arc consistency (AC)** — every value in D_i has a **support** in every other D_j such that c holds.
- **Bound consistency (BC)** — the same, but only the endpoints $\min(D_i), \max(D_i)$ need a support, and the supporting values can be any real in $[\min, \max]$.
- **Path / k-consistency** — stronger, more expensive, rarely worth it.

Enforcing AC removes more values than BC, but costs more per call. Most modern solvers (including NuCS) lean on BC + smart global constraints.

AC-3, the canonical propagation algorithm

```
Q ← all (variable, constraint) pairs
while Q not empty:
    (x, c) ← Q.pop()
    if revise(x, c) reduced D_x:
        if D_x is empty: return INCONSISTENT
        Q ← Q ∪ { (y, c') : y is a neighbor of x in c' ≠ c }
return CONSISTENT
```

Key idea: **only re-wake constraints whose variables changed.**

In NuCS this becomes the *triggered_propagators* queue + per-variable **event masks** (MIN changed, MAX changed, GROUND).

Global constraints: more than syntactic sugar

`all_different(x_1, ..., x_n)` is logically equivalent to a clique of $\binom{n}{2}$ pairwise $\neq$ constraints.

But:

- The clique form propagates **weakly**: each $\neq$ only fires when one side is ground.
- A dedicated `all_different` propagator can reason about the **whole set**: e.g. Régin's AC algorithm uses bipartite matching; bound-consistent versions use Hall intervals.

The same logic, **more reasoning per call** → much smaller search tree.

This is why CP scales on combinatorial problems.

Search: backtracking + branching heuristics

```
search(state):  
    propagate(state)  
    if inconsistent: return FAIL  
    if all variables ground: yield SOLUTION  
    x ← choose_variable(state)           # variable heuristic  
    for v in choose_values(D_x):         # value heuristic  
        push(state); assign(x = v)  
        search(state)  
        pop(state)
```

Heuristic quality is **everything**. Common rules:

- **Variable**: smallest domain first (*first-fail*); most-constrained; impact-based.
- **Value**: smallest first; min-conflicts; promise-based.

Optimization = repeated satisfaction

To minimize an objective variable z :

```
best ← +∞
while True:
    add temporary constraint  $z < \text{best}$ 
    sol ← solve()           # one solution
    if sol is None: return best
    best ← sol[z]
```

Each new solution tightens the upper bound on z . The added bound prunes the rest of the tree.

This is sometimes called **branch and bound**.

NuCS: `solver.minimize(var)` / `solver.maximize(var)` .

Part 3 — Real problems, in NuCS

Problem 1 — N-Queens (revisited)

Three lines, again:

```
super().__init__([(0, n - 1)] * n)
self.add_propagator(ALG_ALLDIFFERENT, range(n))
self.add_propagator(ALG_ALLDIFFERENT, range(n), range(n)) # x_i + i all different
self.add_propagator(ALG_ALLDIFFERENT, range(n), range(0, -n, -1)) # x_i - i all different
```

The clever bit: the second/third `add_propagator` pass an **offset** per variable.

`all_different` is applied to $x_i + i$ (resp. $x_i - i$) without materializing those expressions — the offset is baked into the propagator.

That's three "reasoning hubs", not $3 \cdot \binom{n}{2}$ disequalities.

Problem 2 — Sudoku

```
class SudokuProblem(LatinSquareProblem):
    def __init__(self, givens):
        super().__init__(list(range(1, 10)), givens)
        for i in range(3):
            for j in range(3):
                o = i * 27 + j * 3
                self.add_propagator(ALG_ALLDIFFERENT,
                                    [0+o, 1+o, 2+o, 9+o, 10+o, 11+o, 18+o, 19+o, 20+o])
```

- 81 variables, each with domain $\{1, \dots, 9\}$.
- `LatinSquareProblem` provides 9 row + 9 column `all_different` s.
- Add 9 `all_different` s for the 3×3 boxes.

Hardest Sudokus → milliseconds.

Problem 3 — Cryptarithmic (SEND + MORE = MONEY)

Variables: one per distinct letter, domain $\{0, \dots, 9\}$.

Constraints:

- `all_different(S, E, N, D, M, O, R, Y)`
- $S \neq 0, M \neq 0$ (no leading zero)
- A linear equation summing $1000 \cdot S + 100 \cdot E + \dots$

NuCS expresses this with `ALG_AFFINE_EQ` (a single linear-equality propagator) + one `ALG_ALLDIFFERENT`.

Notice the pattern: **few constraints, each doing a lot.**

Problem 4 — Golomb ruler (an optimization problem)

Find n integer marks $0 = m_0 < m_1 < \dots < m_{n-1}$ such that all pairwise distances $m_j - m_i$ are distinct and m_{n-1} is **minimal**.

NuCS uses a distance-variable model:

```
# one variable per pair (i, j) → dist_ij = m_j - m_i
self.add_propagator(ALG_SUM_EQ, [d_0i, d_ij, d_0j]) # for each i<j
self.add_propagator(ALG_ALLDIFFERENT, range(dist_nb))
```

Plus **redundant constraints** that the solver could deduce in principle, but doesn't, fast enough:

$\text{dist}_{i,j} \leq m_{n-1} - \text{sum of first } (n - 1 - (j - i)) \text{ integers.}$

Redundant constraints are a **major** practical lever.

Golomb: symmetry breaking

The reversed ruler is also a Golomb ruler. To avoid exploring it:

```
# first gap < last gap  
self.add_propagator(ALG_LEQ_C, [d_{0,1}, d_{n-2,n-1}], [-1])
```

One linear constraint = the search tree halved.

Symmetry breaking is a recurring CP technique — exploit problem structure to forbid equivalent solutions.

Problem 5 — TSP

n cities, costs c_{ij} . Find a Hamiltonian cycle of minimum total cost.

Modeling idea — **successor representation**:

- $\text{succ}_i \in \{0, \dots, n - 1\} \setminus \{i\}$ — the next city after i .
- A global **circuit** constraint enforces that succ forms one Hamiltonian cycle (no sub-tours).
- For each i , the cost contribution $\text{cost}_i = c_{i, \text{succ}_i}$ — modeled with **element** : index into the row c_i by the variable succ_i .

TSP in NuCS

```
class TSPProblem(CircuitProblem):
    def __init__(self, costs):
        n = len(costs)
        super().__init__(n) # adds the circuit constraint over succ/pred
        self.succ_costs = self.add_variables([...])
        self.total_cost = self.add_variable((sum_min, sum_max))
        for i in range(n):
            self.add_propagator(ALG_ELEMENT_EQ,
                               [i, self.succ_costs + i], costs[i]) # cost_i = costs[i][succ_i]
        self.add_propagator(ALG_SUM_EQ,
                            list(range(self.succ_costs, self.succ_costs + n)) + [self.total_cost])
```

Then: `solver.minimize(problem.total_cost)` .

Lessons from the four examples

1. **Few high-level constraints beat many primitive ones.**
2. **Reformulation matters** — successor variables for TSP, distance variables for Golomb, dual models for Queens.
3. **Redundant constraints** strengthen propagation without changing the solution set.
4. **Symmetry breaking** divides the search tree.
5. **The objective variable participates in propagation** — that's the engine of branch-and-bound.

Part 4 — Inside the solver

Anatomy of a CP solver

Domains (state)	– [min, max] per variable
Propagators	– constraint implementations
Trigger table	– variable × event → propagator set
Propagation queue	– propagators waiting to re-fire
Search engine	– branching + backtracking
Heuristics	– choose variable, choose value
Statistics	– backtracks, props, solutions, ...

NuCS specifics: everything is a **NumPy array** so it can be JIT-compiled with **Numba**.

Propagator interface (NuCS)

Each propagator is **three functions** and an entry in a registry:

```
ALG_X = register_propagator(get_triggers_x, get_complexity_x, compute_domains_x)
```

- `get_triggers_x(n, variable, parameters) -> EVENT_MASK` — *which* domain events of *this* variable should re-wake the propagator?
- `get_complexity_x(n, parameters) -> int` — used to order the propagation queue (cheap first).
- `compute_domains_x(domains, parameters) -> {INCONSISTENCY, CONSISTENCY, ENTAILMENT}` — the actual filtering. Returns whether we hit a contradiction, made progress, or the constraint is now permanently satisfied (entailed).

A real propagator: `abs(y) = x`

```
@njit(cache=True, fastmath=True)
def compute_domains_abs_eq(domains, parameters):
    y, x = domains[0], domains[1]
    if y[MIN] > 0:                                # y already positive → abs(y) = y
        if y[MIN] > x[MIN]: x[MIN] = y[MIN]
        if y[MAX] < x[MAX]: x[MAX] = y[MAX]
        ...
    elif y[MAX] < 0:                              # y already negative → abs(y) = -y
        ...
    else:                                         # 0 ∈ [y[MIN], y[MAX]]
        if x[MIN] < 0: x[MIN] = 0
        max_y = max(-y[MIN], y[MAX])
        if max_y < x[MAX]: x[MAX] = max_y
        ...
    return PROP_CONSISTENCY
```

Pure mutation of `min/max`. No allocation. No Python objects. JIT-compiled.

Events and triggers — only re-wake what matters

```
def get_triggers_abs_eq(n, variable, parameters):  
    if variable == 0:    # y: changing either bound affects x  
        return EVENT_MASK_MIN_MAX  
    return EVENT_MASK_MAX    # x: only MAX shrinking can prune y
```

Event masks: MIN changed, MAX changed, GROUND reached.

A propagator subscribes to a precise subset.

Wakeup discipline is half the performance story — the other half is keeping the work itself fast.

The propagation loop (pseudocode)

```
while queue not empty:
    p ← queue.pop_lowest_complexity()
    status ← p.compute_domains(domains[p.vars], p.params)
    if status == INCONSISTENCY:
        return FAIL
    if status == ENTAILMENT:
        mark p as entailed (skip in this subtree)
    for v in p.vars where domain changed:
        for (event, q) in triggers[v]:
            if event in changes[v]:
                queue.add(q)
```

Fixed-point. Terminates because domains are finite and only shrink.

Search + propagation, together

```
def search(state, depth):
    propagate(state)                    # fixed-point
    if FAIL:                            backtrack(); return
    if all ground:                      yield solution; return
    x ← choose_variable(state)
    for branch in choose_values(D_x):    # e.g. x = v    |    x ≠ v
        push_state(state)               # save min/max + entailed flags
        assert branch                   # adds to the queue
        search(state, depth+1)
    pop_state(state)
```

Two engines, one loop. The **trail** (saved domain snapshots) is what makes backtracking $O(\text{vars})$.

Why this is hard to make fast in Python

- Per-variable updates are **tight integer math**: comparisons, mins, maxes.
- A solve can do **millions** of propagator calls per second.
- Pure Python's interpreter overhead would dominate by 50–100×

NuCS strategy:

1. **Numba JIT** every hot function (`@njit(cache=True, fastmath=True)`).
2. **No Python objects in jitted code** — only typed NumPy arrays and ints.
3. **Mutate in place**, never reallocate.
4. **Function pointers via** `_get_wrapper_address` so propagators can be dispatched from jitted code.
5. **Multiprocessing** for embarrassingly parallel splits (`MultiprocessingSolver` over `problem.split(...)`).

What you give up vs. what you get

You give up:

- Sparse domains with holes (NuCS is bound-consistent).
- Arbitrary Python in hot paths.
- Some expressiveness — you model with the propagators that exist (or write a new one).

You get:

- A **pure-Python** package that's competitive with C++ solvers on many benchmarks.
- A small, readable codebase you can extend.
- Numba's warm-cache startup, plus trivial multiprocessing parallelism.

When should you reach for CP?

✓ Good fit:

- Combinatorial puzzles, **scheduling**, **rostering**, configuration, routing.
- Problems with rich logical structure: `all_different`, precedences, packing.
- Optimization where the objective is **integer** and the model is highly constrained.

⚠ Less good fit:

- Pure linear / continuous → use **MIP**.
- Boolean-heavy with no structure → **SAT** may be faster.
- Very-large-scale industrial OR → **CP-SAT (OR-Tools)** combines ideas from CP + SAT.

What we covered

1. NP-completeness motivates **smart search**, not just enumeration.
2. $CP = \langle X, D, C \rangle + \text{propagation} + \text{search}$.
3. Global constraints (`all_different`, `circuit`, `element`, ...) carry most of the reasoning.
4. NuCS models problems in a few `add_propagator` calls.
5. A solver is a propagation queue + a backtracking loop; NuCS makes it fast with Numba.

Resources

- **CSPLIB** — <https://www.csplib.org> — reference catalogue of problems (Queens = #54, Golomb = #6).
- **Handbook of Constraint Programming** — Rossi, van Beek, Walsh (Elsevier, 2006).
- **MiniZinc Handbook** — best on-ramp to modeling.
- **NuCS** — <https://github.com/yangeorget/nucs>
- **OR-Tools CP-SAT** — Google's industrial-grade solver.

Questions?

Bonus demo material if there's time: live-run Queens, Sudoku, Golomb-10, TSP-15.